Peak vs Average Traffic

how to survive a 50× spike

Watch the 3-min video

In one line: users feel the peak, not the average. Size for the expected peak plus 30–50% headroom, and absorb spikes with a CDN, a waiting room, queues, pre-scaling, rate limiting and graceful load shedding.

1 The 10 o'clock problem

A concert ticket sale opens at 10:00. On a normal day your site handles **2,000 requests per second**. At 9:59, a million fans are refreshing the page. At 10:00, traffic jumps to **100,000 requests per second**: fifty times normal. If you sized for the average, the site is already down.

2 Traffic is never flat

Most apps follow a **daily (diurnal) pattern**: quiet at night, rising in the morning, peaking in the evening. The busiest hour is often **2–3× the daily average**. Weekly and seasonal patterns stack on top: Monday mornings for business apps, Black Friday for shops.

Then there are **spikes**, or flash crowds: a ticket sale, a product launch, a celebrity post, a push notification sent to every user at once, breaking news. They can be **10–100× normal**, and they arrive in seconds, faster than autoscaling can react (starting new servers usually takes minutes).

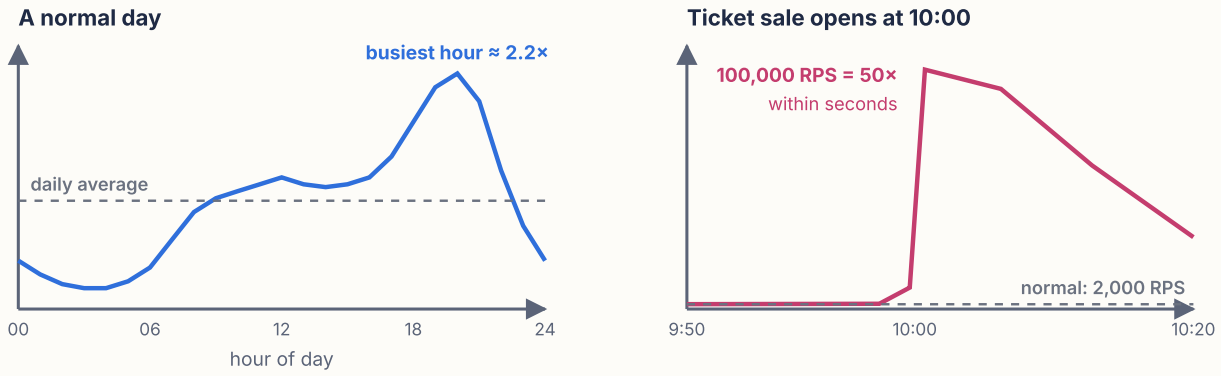


Figure 1. Left: the daily pattern; the busiest hour is about 2–3× the average. Right: a flash crowd; 50× in seconds, which no daily pattern predicts.

3 Why design for peak, not average

Because **users experience the peak**. Averages hide the moment when queues fill up, latency explodes and requests time out. A system at 100% utilization has no room to absorb a burst, and as it nears 100%, waiting time climbs steeply (see SD-A-009 on throughput). Once requests time out, clients retry, adding even more load: a **cascading failure**.

So size for the **expected peak plus headroom**, typically 30–50%. With 50% headroom, servers sit at about two-thirds busy at peak, in the 60–70% range most teams aim for.

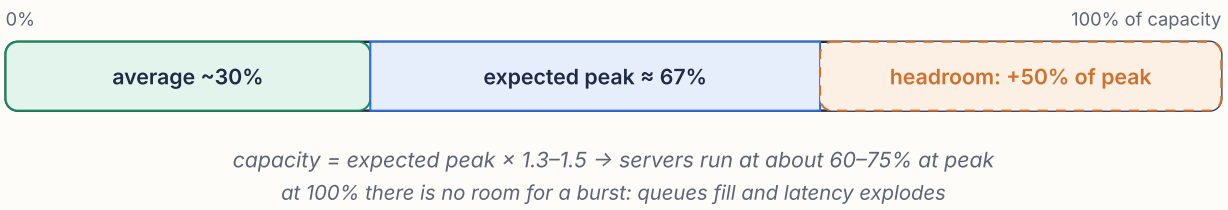


Figure 2. Average, peak and headroom on one capacity bar. The headroom is for the burst you did not predict and the server you just lost.

4 How to absorb a 50× spike

First, don't do the slow work synchronously. Put a **queue** between the front door and the slow part. Accept the request, enqueue it, reply “we're on it”, and process it at the rate the backend can sustain. A **virtual waiting room** does the same for users: let people in at a controlled rate, and show everyone else their place in line.

Then add more layers of defense:

- **CDN for static content.** Serve the event page from the edge, so origin servers never see most requests.
- **Pre-scale before a known event**, instead of waiting for autoscaling to notice the spike.
- **Rate limit per user** (429 Too Many Requests) so one client cannot refresh 50 times a second.
- **Shed load gracefully**: return “try again later” (503 with Retry-After) instead of crashing.
- **Turn off non-essential features** during the peak, such as recommendations or reviews.

5 Worked example: the ticket sale

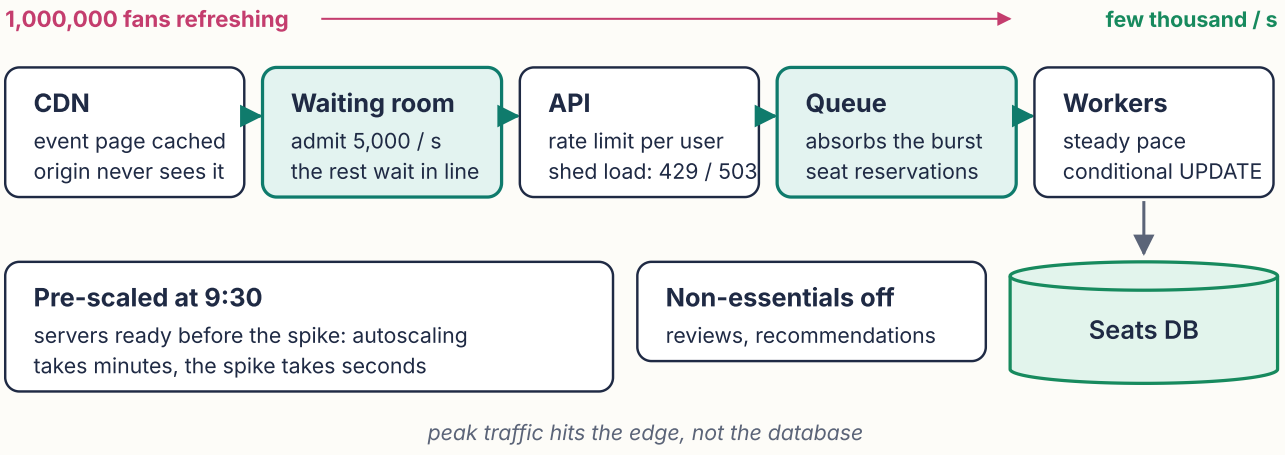


Figure 3. Each layer cuts the traffic that reaches the next. A million refreshing fans become a few thousand writes per second at the database.

WORKED EXAMPLE

- The **event page** is cached at the CDN, so the 100,000 RPS of page loads hit the edge.
- Fans enter a **waiting room** and are admitted at **5,000 per second**.
- Each **seat reservation goes into a queue**; workers process it with a row lock or a conditional update (`… WHERE seat_id = ? AND status = 'free'`), so nothing is oversold.
- The servers were **scaled up at 9:30**, before the sale opened.

Peak traffic hits the edge, not the database.

COMMON MISTAKES

- Sizing capacity from the daily average.
- Relying on autoscaling for a spike that arrives in seconds.
- Doing slow work (payments, seat allocation) inside the request instead of a queue.
- No rate limits or load shedding, so overload means a crash instead of “try again later”.
- Retrying immediately on timeout, which multiplies the spike.

6 Key terms

Diurnal pattern: daily rise and fall of traffic

Peak factor: peak ÷ average

Flash crowd: sudden spike, 10–100× normal

Headroom: spare capacity above the expected peak

Virtual waiting room: admits users at a controlled rate

Load shedding: rejecting some work to keep the rest healthy

Pre-scaling: adding capacity before a known event

Cascading failure: overload and retries spread from one part to all

Rate limiting: capping requests per user or key

7 Interview questions

Q1. Why design for peak rather than average? BASIC

Users feel the peak. A system sized for the average fails exactly when traffic is highest: queues fill, requests time out, retries add load and failures cascade. Size for the peak plus 30–50% headroom.

Q2. What is a typical peak factor? BASIC

For daily patterns, the busiest hour is often 2–3× the daily average. Flash crowds (launches, ticket sales, push notifications) can reach 10–100×.

Q3. Why not run servers at 100% at peak? BASIC

There is no room for a burst or a lost server, and latency rises steeply near full utilization. Teams aim for about 60–70% at peak.

Q4. How would you absorb a 50× spike? SCENARIO

CDN for static content, a waiting room and queues to smooth the burst, pre-scaling before known events, per-user rate limiting, graceful load shedding (503 + Retry-After), and turning off non-essential features.

Q5. Average traffic is 2,000 RPS, the daily peak factor is 3, and you want 50% headroom. How much capacity? SCENARIO

Peak = 2,000 × 3 = 6,000 RPS. With 50% headroom, 6,000 × 1.5 = 9,000 RPS. If one server handles 1,000 RPS, that is 9 servers, plus at least one spare for failures.

Q6. A push notification to 20 M users causes a spike every time. What do you change? SCENARIO

Send it in batches over a few minutes instead of all at once, add jitter on the client before it fetches, and serve the target content from the CDN or cache.

Q7. Why can't autoscaling handle a ticket sale on its own? TRICKY

It reacts to metrics that lag, and new servers take minutes to boot and warm up. A flash crowd arrives in seconds, so the site is overloaded before new capacity is ready. Pre-scale and use a waiting room.

Q8. Isn't returning errors during a spike just failing? TRICKY

Shedding some requests with a quick “try again later” keeps the system healthy for everyone already admitted. Without it, all requests slow down, time out and retry, and nobody gets served.

Next: SD-D-005 · Read/Write Ratio